

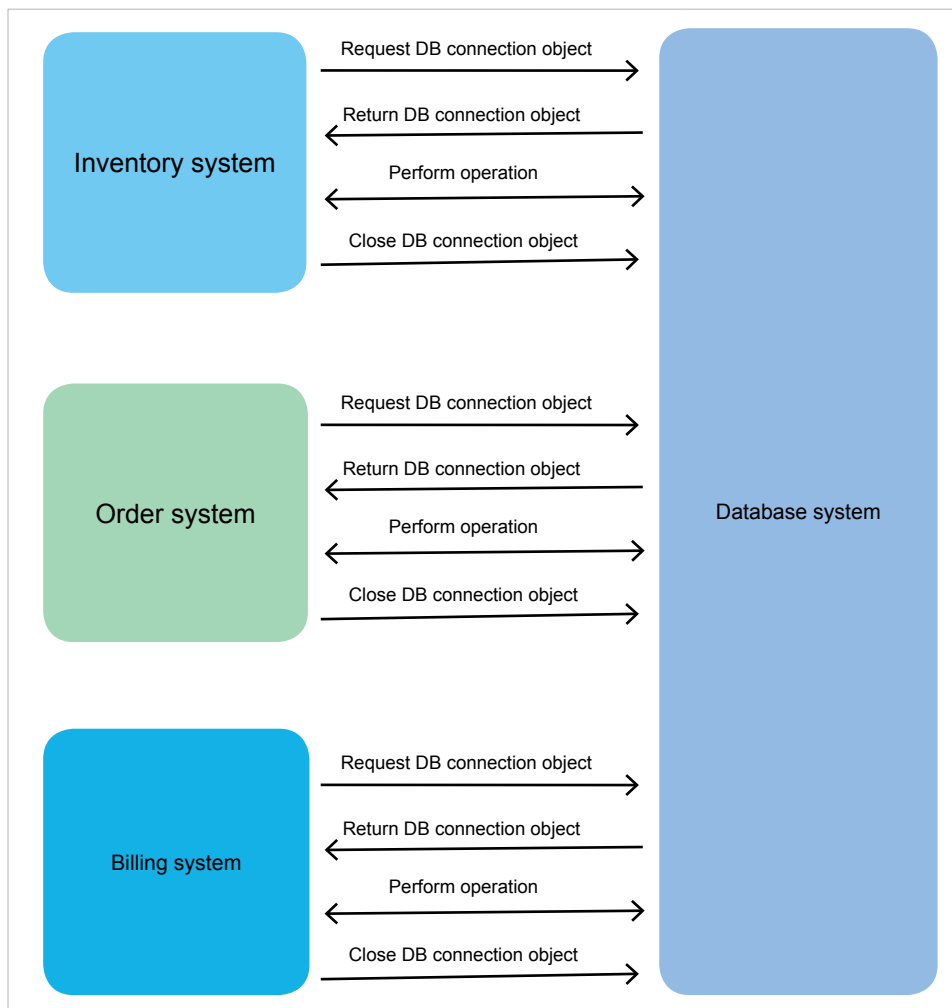


Figure 2: Connection pools

In LRU, the connection pool manager keeps track of the time that each connection was last used. When a new connection is required, the connection pool manager selects the least recently used connection from the pool and returns it to the user.

In FIFO, the connection pool manager manages connections in the order they were added to the pool. When a new connection is required, the connection pool manager selects the connection that has been in the pool the longest and returns it to the user.

Connection pooling configurations are used to set the parameters for the connection pool. These configurations include settings such as the minimum and maximum number of connections in the pool, the maximum time a connection can be idle before it is closed, and the maximum time a connection can be used before it is returned to the pool.

Overall, the basic principles of connection pooling involve creating a pool of database connections, managing the

pool using algorithms and configurations, and reusing the connections as required to reduce overhead and improve performance.

Implementing our own connection pool

To implement connection pooling in a specific programming language or framework, developers typically use connection pool libraries or built-in connection pool features. Code snippets and examples for implementing connection pools are often available in library documentation or online resources.

However, simply integrating an existing library in some dummy application is no good for us. Additionally, as software engineers, implementing our own connection pool can bring a wealth of knowledge benefits. First, it can significantly improve the performance of our application by reducing the overhead associated with establishing new

connections. Additionally, it can help to prevent connection leaks and other issues that can arise from improperly managed connections.

Moreover, it provides us with fine-grained control over connection creation, usage, and destruction, allowing us to optimise our application's resource utilisation. By implementing our own connection pooling, we can gain a deeper understanding of how our application works and thereby improve its scalability and reliability.

Building blocks

For ease of demonstration, we can use an SQLite3 database and implement our own custom pooling for it. In this example, we'll be using the Go programming language due to its simplicity. However, you can use any language of your choice.

To start with, our *ConnectionPool struct* will look something like this: